

to hold the digits of π . We write a loop that adds each line of digits to `pi_string` ❶. We print this string, and also show how long the string is:

```
3.1415926535 8979323846 2643383279
36
```

The variable `pi_string` contains the whitespace that was on the left side of the digits in each line, but we can get rid of that by using `rstrip()` on each line:

```
--snip--
for line in lines:
    pi_string += line.rstrip()

print(pi_string)
print(len(pi_string))
```

Now we have a string containing π to 30 decimal places. The string is 32 characters long because it also includes the leading 3 and a decimal point:

```
3.141592653589793238462643383279
32
```

NOTE

When Python reads from a text file, it interprets all text in the file as a string. If you read in a number and want to work with that value in a numerical context, you'll have to convert it to an integer using the `int()` function or a float using the `float()` function.

Large Files: One Million Digits

So far, we've focused on analyzing a text file that contains only three lines, but the code in these examples would work just as well on much larger files. If we start with a text file that contains π to 1,000,000 decimal places, instead of just 30, we can create a single string containing all these digits. We don't need to change our program at all, except to pass it a different file. We'll also print just the first 50 decimal places, so we don't have to watch a million digits scroll by in the terminal:

```
pi_string.py from pathlib import Path

path = Path('pi_million_digits.txt')
contents = path.read_text()

lines = contents.splitlines()
pi_string = ''
for line in lines:
    pi_string += line.rstrip()

print(f"{pi_string[:52]}...")
print(len(pi_string))
```
